



**Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)**

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3
по курсу «Алгоритмы компьютерной графики»
«Введение в программирование трёхмерной графики»

Студент группы ИУ9-42Б Ивенкова О. В.

Преподаватель Цалкович П. А.

Москва 2025

1 Задача

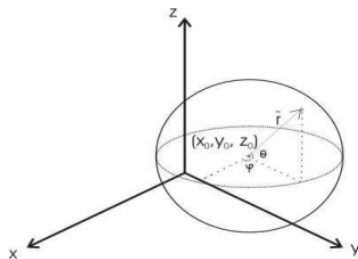
- 1. Определить параметризованную модель объекта сцены (персональный вариант: эллипсоид).
- 2. Определить преобразования, позволяющие получить заданный вид проекции (в соответствии с вариантом). Для демонстрации проекции добавить в сцену куб (в стандартной ориентации, не изменяемой при модельно-видовых преобразованиях основного объекта).
- 3. Реализовать изменение ориентации и размеров объекта (навигацию камеры) с помощью модельно-видовых преобразований (без gluLookAt). Управление производится интерактивно с помощью клавиатуры и/или мыши.
- 4. Предусмотреть возможность переключения между каркасным и твердотельным отображением модели (glFrontFace/ glPolygonMode).

2 Основная теория

Параметрическое задание поверхности

- координаты точек поверхности рассматриваются как некие функции от двух параметров, пробегающих некоторый набор значений;
- используются различного рода особые параметрические поверхности, например, поверхности Безье, В-сплайны, в частности, NURBS (Non-Uniform Rational B-Spline, неоднородный рациональный В-сплайн).

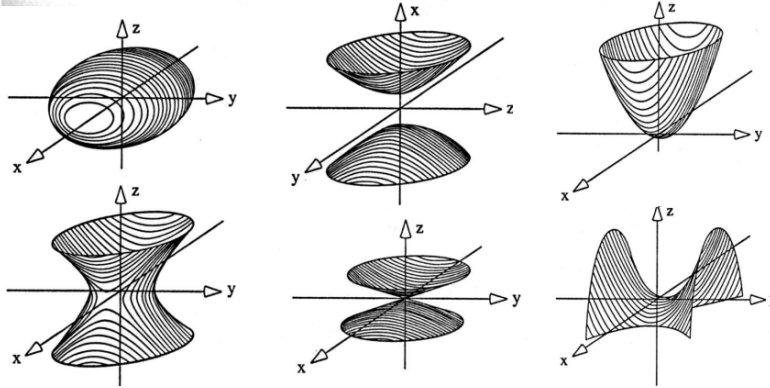
$$\begin{cases} x = x(u, v) \\ y = y(u, v) \\ z = z(u, v) \end{cases}$$



$$\begin{cases} x = x_0 + r \cos \theta \cos \varphi \\ y = y_0 + r \cos \theta \sin \varphi, \quad 0 \leq \varphi < 2\pi, \quad -\frac{\pi}{2} \leq \theta \leq \frac{\pi}{2} \\ z = z_0 + r \sin \theta \end{cases}$$

Поверхности второго порядка (квадрики)

Название квадрики	Неявная форма	Параметрическая форма	Интервал изменения по v и по u
Эллипсоид	$x^2 + y^2 + z^2 = 1$	$(\cos(v) \cos(u), \cos(v) \sin(u), \sin(v))$	$(-\pi/2, \pi/2), (-\pi, \pi)$
Однополостный гиперболоид	$x^2 + y^2 - z^2 = 1$	$(\sec(v) \cos(u), \sec(v) \sin(u), \tg(v))$	$(-\pi/2, \pi/2), (-\pi, \pi)$
Двуполостный гиперболоид	$x^2 - y^2 - z^2 = 1$	$(\sec(v) \cos(u), \sec(v) \tg(u), \tg(v))$	$(-\pi/2, \pi/2)^1$
Эллиптический конус	$x^2 + y^2 = z^2$	$(v \cos(u), v \sin(u), v)$	Любые вещ. числа, $(-\pi, \pi)$
Эллиптический параболоид	$x^2 + y^2 = z$	$(v \cos(u), v \sin(u), v^2)$	$v \geq 0, (-\pi, \pi)$
Гиперболический параболоид	$-x^2 + y^2 = z$	$(v \tg(u), v \sec(u), v^2)$	$v \geq 0, (-\pi, \pi)$

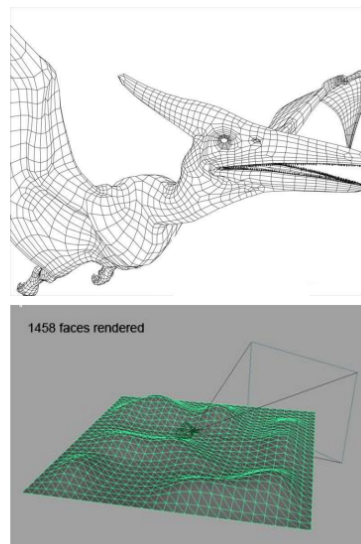


Граничное представление: разбиение на грани

- глобальная параметризация объекта произвольной формы является трудновыполнимой задачей: поверхность приближается набором *граней (face)*, каждая из которых имеет компактное математическое представление;
- границы граней представляются *ребрами (edge)*;
- часть кривой, формирующей ребро, заканчивается *вершинами (vertex)*;
- поверхностная модель, которая состоит исключительно из плоских граней, называется *полигональной моделью (полигональной сеткой, polygonal mesh)*;
- полигональная модель может задавать:
 - многогранники;
 - аппроксимации гладких объектов (линейная интерполяция).

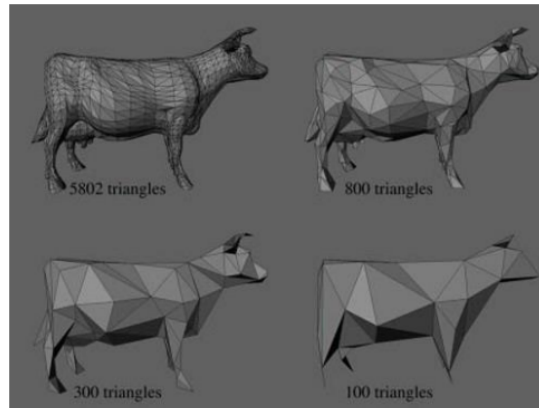
Полигональное представление

- для построения требует информации о поверхности объекта;
- рассматриваются представления первого порядка (линейная интерполяция);
- структура данных:
 - вершины: геометрия (координаты);
 - границы: топология (связность);
 - нормали: ориентация;
- свойства полигональных сеток:
 - монолитность (ограничивает некоторое конечное пространство);
 - связность (существует непрерывный путь вдоль ребер полигона);
 - простота (монолитность и отсутствие отверстий);
 - плоскостность (каждая грань задается плоским многоугольником);
 - выпуклость.



Триангуляция

- триангуляция – выделение троек вершин (~ tessellation / tiling) / аппроксимация поверхности треугольными гранями:
 - грани, образованные в результате триангуляции, заведомо задают плоскости;
 - упрощает выполнение многих функций графических пакетов;
 - для выпуклых многоугольников сводится к последовательному выделению троек вершин.



Разбиение многоугольников в OpenGL

- OpenGL поддерживает задание и отображение простых выпуклых многоугольников (задаваемых единственным несамопересекающимся контуром), для остальных случаев необходимо применять процедуру разбиения (tessellation)
- процедура разбиения
 1. создание объекта разбиения: **gluNewTess()**;
 2. определение функций обратного вызова для процедуры разбиения: **gluTessCallback()**
 - определение наборов возвращаемых данных (GL_TRIANGLE_FAN, GL_TRIANGLE_STRIP, GL_TRIANGLES, GL_LINE_LOOP);
 - результат разбиения в виде последовательности вершин;
 - вычисление атрибутов дополнительных вершин в случае самопересечений;
 - информация об ошибках;
 3. определение режима разбиения: **gluTessProperty()**
 - GLU_TESS_BOUNDARY_ONLY – получение границ результирующих областей;
 - GLU_TESS_TOLERANCE – точность операций с вершинами (анализа на совпадение);
 - GLU_TESS_WINDING_RULE – определение правила задания внутренних и внешних областей (GLU_TESS_WINDING_ODD – по умолчанию, GLU_TESS_WINDING_NONZERO, GLU_TESS_WINDING_POSITIVE, GLU_TESS_WINDING_NEGATIVE, GLU_TESS_WINDING_ABS_GEQ_TWO);
 4. задание набора многоугольников через набор контуров
 - **gluTessBeginPolygon () / gluTessEndPolygon();**
 - **gluTessBeginContour () / gluTessEndContour();**
 - **gluTessVertex ();**
 5. удаление объекта разбиения: **gluDeleteTess()**.

3 Практическая реализация

Исходный код программы представлен в листинге 1.

Листинг 1: Реализация файла main.py

```
1 import glfw
2 from OpenGL.GL import *
3 from OpenGL.GLUT import *
4 from OpenGL.GLU import *
5 import numpy as np
6
7 angle_x, angle_y = 0, 0
8 scale = 0.7
9 wireframe = False
10
11 a, b, c = 1.0, 0.6, 0.8
12 num_lat = 10
13 num_lon = 20
14
15 def parametric_ellipsoid(theta, phi):
16     x = a * np.cos(theta) * np.cos(phi)
17     y = b * np.sin(theta)
18     z = c * np.cos(theta) * np.sin(phi)
19     return x, y, z
20
21 def draw_parametric_ellipsoid():
22     glBegin(GL_TRIANGLES)
23
24     for i in range(num_lat):
25         theta1 = -np.pi / 2 + np.pi * i / num_lat
26         theta2 = -np.pi / 2 + np.pi * (i + 1) / num_lat
27
28         for j in range(num_lon):
29             phi1 = 2 * np.pi * j / num_lon
30             phi2 = 2 * np.pi * (j + 1) / num_lon
31
32             v1 = parametric_ellipsoid(theta1, phi1)
33             v2 = parametric_ellipsoid(theta2, phi1)
34             v3 = parametric_ellipsoid(theta2, phi2)
35             v4 = parametric_ellipsoid(theta1, phi2)
36
37             glVertex3fv(v1)
38             glVertex3fv(v2)
39             glVertex3fv(v3)
40
41             glVertex3fv(v1)
42             glVertex3fv(v3)
43             glVertex3fv(v4)
44
45     glEnd()
```

```

46
47 def draw_moving_ellipsoid():
48     glMatrixMode(GL_MODELVIEW)
49     glLoadIdentity()
50     glRotatef(angle_x, 1, 0, 0)
51     glRotatef(angle_y, 0, 1, 0)
52
53     glPushMatrix()
54     glScalef(scale, scale, scale)
55     glColor3f(75/255, 0, 130/255)
56     draw_parametric_ellipsoid()
57     glPopMatrix()
58
59 def display():
60     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
61     draw_moving_ellipsoid()
62     glfw.swap_buffers(window)
63
64 def key_callback(window, key, scancode, action, mods):
65     global angle_x, angle_y, wireframe, scale
66     if action == glfw.PRESS or action == glfw.REPEAT:
67         if key == glfw.KEY_UP:
68             angle_x -= 5
69         elif key == glfw.KEY_DOWN:
70             angle_x += 5
71         elif key == glfw.KEY_LEFT:
72             angle_y -= 5
73         elif key == glfw.KEY_RIGHT:
74             angle_y += 5
75         elif key == glfw.KEY_EQUAL:
76             scale += 0.1
77         elif key == glfw.KEY_MINUS:
78             scale -= 0.1
79         elif key == glfw.KEY_W:
80             wireframe = not wireframe
81         glPolygonMode(GL_FRONT_AND_BACK, GL_LINE if wireframe else
GL_FILL)
82
83 def scroll_callback(window, xoffset, yoffset):
84     global scale
85     scale += 0.1 if yoffset > 0 else -0.1
86
87 def init():
88     glEnable(GL_DEPTH_TEST)
89
90 if not glfw.init():

```

```

91     raise Exception("GLFW can't be initialized")
92
93 window = glfw.create_window(800, 600, "lab3", None, None)
94 if not window:
95     glfw.terminate()
96     raise Exception("GLFW window can't be created")
97
98 glfw.make_context_current(window)
99 glfw.set_key_callback(window, key_callback)
100 glfw.set_scroll_callback(window, scroll_callback)
101 init()
102
103 while not glfw.window_should_close(window):
104     display()
105     glfw.poll_events()
106
107 glfw.terminate()

```

Результат запуска представлен на рисунках 1– 2.

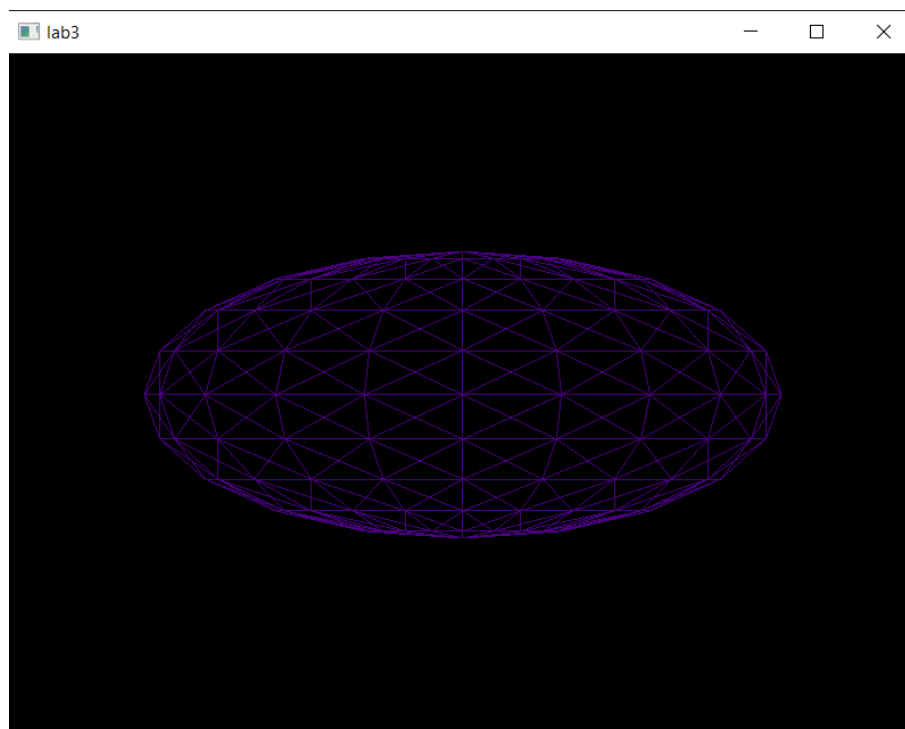


Рис. 1 — Каркасное отображение эллипсоида

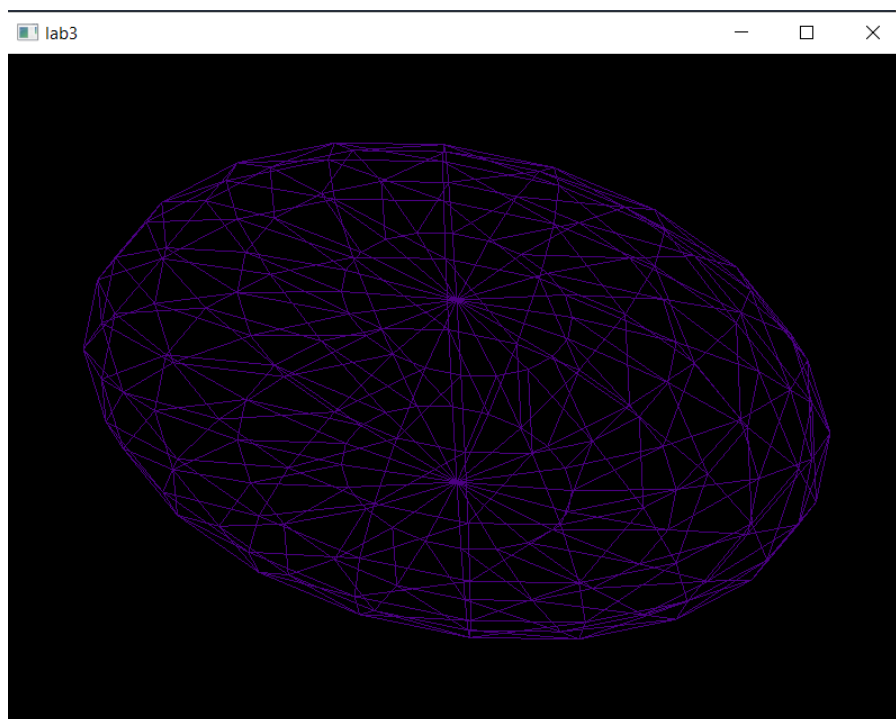


Рис. 2 — Повёрнутый и увеличенный эллипсоид

4 Заключение

В ходе выполнения работы была разработана параметризованная модель эллипсоида, заданная уравнениями в сферических координатах. Для визуализации фигуры реализовано полигональное разбиение, разбивающее поверхность эллипсоида на треугольные грани, что обеспечивает его корректное отображение.

Реализована возможность изменения ориентации и масштаба эллипсоида с помощью модельно-видовых преобразований ('glRotatef', 'glScalef'). Управление осуществляется интерактивно с помощью клавиатуры и мыши, что позволяет пользователю вращать объект и изменять его размер в реальном времени.

Кроме того, предусмотрен режим переключения между каркасным и твердотельным отображением с использованием 'glPolygonMode()'. Это позволяет анализировать как внешний вид фигуры, так и ее структуру полигонального разбиения.

Таким образом, поставленные задачи были успешно выполнены: создана параметрическая модель эллипсоида, реализовано полигональное разбиение, добавлено интерактивное управление и предусмотрен режим отображения каркасной модели.